

Azure Networking — Hands-on Lab

Virtual Networks: subnets, NSGs, peering, and load balancing

September 2026

Contents

Introduction & Setup	2
Hands-on Lab Exercises: Azure Networking	2
Part 1 — Lab Questions	6
Question 1 — Create a Virtual Network and Subnets	6
Question 2 — Place a VM in a Subnet	7
Question 3 — Secure a Subnet with an NSG	8
Question 4 — Peer Two Virtual Networks	9
Question 5 — Load-Balance Two VMs	10
Part 2 — Solutions	11
Question 1 — Create a Virtual Network and Subnets	11
Solution — Create a Virtual Network and Subnets	11
Question 2 — Place a VM in a Subnet	13
Solution — Place a VM in a Subnet	13
Question 3 — Secure a Subnet with an NSG	15
Solution — Secure a Subnet with an NSG	15
Question 4 — Peer Two Virtual Networks	17
Solution — Peer Two Virtual Networks	17
Question 5 — Load-Balance Two VMs	19
Solution — Load-Balance Two VMs	19
Part 3 — Practice Exercises	22
Practice Exercises — Networking	22

Introduction & Setup

Hands-on Lab Exercises: Azure Networking

Topic: Building and connecting private networks in Azure with **Virtual Networks:** subnets, network security groups, VNet peering, and load balancing.

Welcome to the third lab in the Cloud Administrator series. Lab 1 was about **compute** (Virtual Machines and App Service) and Lab 2 was about **data** (Blob Storage and Azure SQL). This lab is about the **network** those resources live on and talk across.

A real admin does not memorize every command. They know which breadcrumbs to follow. Each problem here is a short, real world scenario. Your job is to work out what it demands and solve it with the Azure CLI. You get a script to check your work and a concise solution walkthrough, freely, because this is a low stakes place to build competency.

What you will build

One category, five graded questions, plus five practice exercises:

- Create a **virtual network** and carve it into **subnets**.
- Place a **VM** into a subnet and see it pick up a private IP from that range.
- Control traffic with a **network security group** on a subnet.
- Connect two VNets with **peering** so their VMs can reach each other privately.
- Put a **public load balancer** in front of two VMs so one address spreads traffic across both.

How this lab is different from the earlier labs

In the compute and storage labs you mostly managed one resource at a time. Here the subject is the **space between** resources: how they get addresses, who is allowed to reach whom, and how separate networks are joined. A few things follow from that:

- **You see connectivity, not just configuration.** Most questions ask you to `curl` or connect across a boundary and watch it change: a request that times out before a rule and returns 200 after, or a private IP that is unreachable until two networks are peered.
- **The lab uses small “probe” VMs.** To prove that traffic does or does not flow, several questions provision one or two tiny VMs to test from. That means these questions cost more and take a little longer than the storage lab, and a couple of them (peering and load balancing) create two VMs each.
- **Addresses and CIDR ranges matter.** You will work in `10.10.0.0/16` and its `/24` subnets, and checking that a VM landed in the right range is part of the job.

The three skills we are building

1. **Reading documentation.** The first question an admin asks is “where are the provider’s docs for X?” Learn to navigate them and you can learn any service.

2. **Tooling.** We use the **Azure CLI** (**az**) exclusively. No portal for the exercises. We like the hard way, because the hard way is the way that sticks.
3. **Troubleshooting.** When things break, composure first. Understand the cause, fix it, and write down what happened.

Prerequisites

- **An Azure subscription**, paid (pay as you go). The probe VMs use an inexpensive burstable size (Standard_B2als_v2).
- **A lab VM (jumpbox) to work from.** You created this in the Compute lab and it is meant to last the whole semester. Use that same jumpbox here. The Azure CLI runs there, not on your laptop.

No jumpbox yet? Set one up using the “Set up your lab environment” section of the Compute lab README (an Ubuntu 24.04 LTS VM in its own **lab-jumpbox-rg**, with the Azure CLI installed and **az login** done), then come back here.

Region tip. Azure capacity varies by region. The default is **centralus**. If a create fails with a capacity or allocation error, change **LAB_LOCATION** in **lab-config.sh** and try another region such as **eastus2**, **westus2**, **westus3**, **southcentralus**, **westeurope**, or **northeurope**.

Getting started on your jumpbox

1. **Connect to the jumpbox** (VS Code Remote-SSH gives you a file explorer, a rendered preview of each **problem.md** and **solution.md**, and a terminal; plain **ssh** or the Portal’s Bastion work too).

2. **Clone this repo onto the jumpbox:**

```
git clone https://github.com/sailplatform/cloud-admin-lab-exercises-networking.git
cd cloud-admin-lab-exercises-networking
```

3. **Work each question from inside its folder:**

```
cd 1-virtual-networks/Question-1-Create-VNet-and-Subnets
./setup.bash          # then read problem.md and solve it
```

The probe VMs use generated SSH keys (no password). One question (peering) has you SSH into a VM to run a test from inside the network; **setup.bash** prints the exact **ssh** command and the IP to use.

Cost warning. These labs create **real Azure resources that cost money**. Each question is isolated in its own resource group. Questions 2 through 5 provision VMs, and the peering and load balancer questions create **two** VMs each. You are billed while they exist, so the golden rule is: provision, solve, validate, then **run ./cleanup.bash the moment you finish a question**. Keep your jumpbox for the whole semester, but do not leave question VMs running.

How each question is structured

Every graded question lives in its own self contained folder with five files:

File	What it is
setup.bash	Prepares or pre checks the environment (and, where relevant, provisions the starting network and probe VMs).
problem.md	The scenario and the exact requirements (“done” criteria).
validate.bash	Automated PASS or FAIL checks against your subscription.

File	What it is
<code>solution.md</code>	A concise walkthrough of how an admin reasons about it. Read this if you get stuck.
<code>cleanup.bash</code>	Deletes everything the question created.

All scripts are open source: read them. Seeing how a check maps to an `az network ... --query` is good practice in itself.

The workflow

Everything happens inside a single question folder. `cd` into it and go:

```
cd 1-virtual-networks/Question-1-Create-VNet-and-Subnets

./setup.bash      # 1. Prepare or preflight (and, where relevant, provision the starting network)

# 2. Read problem.md and solve it yourself with the Azure CLI.
#   Stuck? Open solution.md in the same folder.

./validate.bash   # 3. Check your work. Prints PASS or FAIL for each requirement.

./cleanup.bash    # 4. Delete the resources so they stop costing money.
```

Configuration: make it yours

Shared settings live in `lab-config.sh` at the repo root: the **region** (`LAB_LOCATION`, default `centralus`), the resource group prefix (`LAB_RG_PREFIX`, default `lab-network`), and the **size and image** for the probe VMs (`LAB_VM_SIZE`, `LAB_VM_IMAGE`). Change a value once and every script follows, or override for a single run:

```
LAB_LOCATION=westus3 ./validate.bash
```

Available questions

1-virtual-networks/: Virtual Networks

#	Topic	Skill
1	Create a VNet and Subnets	Lay out an address space and split it into subnets
2	Place a VM in a Subnet	Attach a VM to a subnet and read its private IP
3	Secure a Subnet with an NSG	Allow HTTP/SSH on a subnet; curl it before and after
4	Peer Two VNets	Join two networks so their VMs reach each other privately
5	Load-Balance Two VMs	One public IP spreads traffic across two backends

Plus [five practice exercises](#) (a three-tier subnet design, an NSG with a service tag, a route table, static versus dynamic public IPs, and an internal load balancer).

The practice mindset (why solutions are given freely)

First study the worked examples, the graded questions, each with a `solution.md` walkthrough of how an admin reasons through the task. Then drill the practice set (problems only, no solutions) to build muscle memory. The point is not to know how to create a subnet or a peering. It is to prove you can, and to build the habit of finding the answer when you do not already have it.

Part 1 — Lab Questions

Question 1 — Create a Virtual Network and Subnets

Scenario

A team is about to deploy a small two-tier application (a web layer and a data layer) and needs a private network in Azure to run it on. As the cloud administrator, create the **virtual network** and carve it into two **subnets**, one for each tier.

First, two terms: virtual network and subnet

- A **virtual network (VNet)** is your own private slice of IP address space in Azure. You give it a range in CIDR notation, for example `10.10.0.0/16` (which covers `10.10.0.0` to `10.10.255.255`, about 65,000 addresses).
- A **subnet** carves a smaller range out of the VNet, for example `10.10.1.0/24` (256 addresses). You attach resources and security rules to subnets, so splitting a VNet into subnets is how you separate tiers (web, data) and control traffic between them.

Requirements

Everything in `lab-network-vnet01-rg`:

#	Requirement	Value
1	A virtual network	address space <code>10.10.0.0/16</code>
2	A subnet web	<code>10.10.1.0/24</code>
3	A subnet data	<code>10.10.2.0/24</code>

After you build it, list the subnets and see the two ranges sitting inside the VNet's address space. (Walk-through in `solution.md`.)

Work the question

```
./setup.bash          # preflight only - you build everything
# ... create the RG, the VNet with the web subnet, then add the data subnet ...
./validate.bash
./cleanup.bash
```

Question 2 — Place a VM in a Subnet

Scenario

The virtual network from Question 1 is ready. Now the web tier needs a server. As the cloud administrator, deploy a Linux VM **into the web subnet** so it picks up a private IP from that subnet's range.

`setup.bash` provisions the VNet (`lab-vnet`) with the `web` and `data` subnets for you.

The idea

When you create a VM you tell Azure which VNet and subnet to attach its network card to. The VM then gets a **private IP** from that subnet's range. Put it in the `web` subnet (`10.10.1.0/24`) and its private IP will be a `10.10.1.x` address. If you forget the `--vnet-name/--subnet` flags, `az vm create` builds a brand-new network instead, which is the usual “why isn't it in my subnet?” mistake.

Requirements

Everything in `lab-network-vnet02-rg`:

#	Requirement	Value
1	A Linux VM attached to the <code>web</code> subnet	private IP inside <code>10.10.1.0/24</code>

After it boots, look at its private IP and confirm it lands in the `web` range. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash      # provisions the VNet + subnets; you create the VM
# ... create a Linux VM in the 'web' subnet, then check its private IP ...
./validate.bash
./cleanup.bash
```

Question 3 — Secure a Subnet with an NSG

Scenario

A VM running a web server sits in the **web** subnet, but nobody can reach it: with no **network security group (NSG)**, Azure’s default rules deny inbound traffic from the internet. As the cloud administrator, add an NSG that permits SSH and HTTP, and apply it to the whole **web** subnet so every VM in that tier is governed by the same rules.

`setup.bash` provisions the VNet and the VM (nginx on port 80, no NSG).

The idea

An **NSG** is a stateful allow/deny list of traffic rules. You can attach it to a single VM’s network card or, better for a tier, to a **subnet** so every resource in that subnet inherits it. Each rule has a priority, a direction, a protocol, a source, and a destination port. You will allow inbound **22** (SSH) and **80** (HTTP) and let the default “deny everything else” stay in place.

Requirements

Everything in `lab-network-vnet03-rg`:

#	Requirement	Value
1	An NSG associated to the web subnet	allows inbound TCP 80 (and 22)

Before the change, `curl http://<vm-public-ip>` times out; after you associate the NSG, it returns 200. Watch that flip. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash          # provisions the VM (nginx, no NSG); prints its public IP
# ... curl it (blocked), create+associate the NSG, curl again (200) ...
./validate.bash
./cleanup.bash
```

Question 4 — Peer Two Virtual Networks

Scenario

Two teams run in two separate virtual networks, `vnet-a` (10.10.0.0/16) and `vnet-b` (10.20.0.0/16). By default the VNets are completely isolated: a VM in one cannot reach a VM in the other, even over private IPs. The teams now need to talk. As the cloud administrator, connect the two networks with **VNet peering**.

`setup.bash` provisions both VNets and a VM in each: `vm-a` (with SSH open, your way in) and `vm-b` (running nginx, private IP only).

The idea

Peering links two VNets so traffic routes between them over Azure's backbone, using private IPs, as if they were one network. Peering is **directional**: you create it on each side (a-to-b and b-to-a). Only when both halves exist does the connection report **Connected** and traffic actually flow.

Requirements

Everything in `lab-network-vnet04-rg`:

#	Requirement	Value
1	Peering <code>vnet-a</code> to <code>vnet-b</code>	state Connected
2	Peering <code>vnet-b</code> to <code>vnet-a</code>	state Connected

Prove it: from `vm-a`, `curl vm-b`'s private IP. It times out before peering and returns 200 after. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash          # provisions both VNets + vm-a and vm-b; prints their IPs
# ... from vm-a, curl vm-b (blocked) -> peer both directions -> curl again (200) ...
./validate.bash
./cleanup.bash
```

Question 5 — Load-Balance Two VMs

Scenario

Two identical web servers, `vm1` and `vm2`, each serve the app. Right now there is no single address in front of them and no way to spread traffic or survive one VM failing. As the cloud administrator, put a **public load balancer** in front of both so one IP distributes requests across them.

`setup.bash` provisions the VNet and both VMs (nginx, each serving its own hostname, port 80 open).

The idea

A **load balancer** takes traffic on a **frontend** (a public IP + port) and spreads it across a **backend pool** of VMs, sending only to the ones a **health probe** says are healthy. You wire up four things: a frontend (public IP), a backend pool (the two VMs' network cards), a probe (is port 80 answering?), and a rule (frontend 80 to backend 80).

Requirements

Everything in `lab-network-vnet05-rg`:

#	Requirement	Value
1	A public load balancer with a rule	frontend port 80 to backend port 80
2	A health probe	on port 80
3	The backend pool contains both VMs	2 members

Prove it: `curl` the load balancer's public IP several times and watch the reply alternate between `vm1` and `vm2`. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash          # provisions vm1 + vm2 (nginx); you build the load balancer
# ... create the LB, pool, probe, rule; add both VMs; curl the LB IP repeatedly ...
./validate.bash
./cleanup.bash
```

Part 2 — Solutions

Question 1 — Create a Virtual Network and Subnets

Scenario

A team is about to deploy a small two-tier application (a web layer and a data layer) and needs a private network in Azure to run it on. As the cloud administrator, create the **virtual network** and carve it into two **subnets**, one for each tier.

First, two terms: virtual network and subnet

- A **virtual network (VNet)** is your own private slice of IP address space in Azure. You give it a range in CIDR notation, for example `10.10.0.0/16` (which covers `10.10.0.0` to `10.10.255.255`, about 65,000 addresses).
- A **subnet** carves a smaller range out of the VNet, for example `10.10.1.0/24` (256 addresses). You attach resources and security rules to subnets, so splitting a VNet into subnets is how you separate tiers (web, data) and control traffic between them.

Requirements

Everything in `lab-network-vnet01-rg`:

#	Requirement	Value
1	A virtual network	address space <code>10.10.0.0/16</code>
2	A subnet web	<code>10.10.1.0/24</code>
3	A subnet data	<code>10.10.2.0/24</code>

After you build it, list the subnets and see the two ranges sitting inside the VNet's address space. (Walk-through in `solution.md`.)

Work the question

```
./setup.bash          # preflight only - you build everything
# ... create the RG, the VNet with the web subnet, then add the data subnet ...
./validate.bash
./cleanup.bash
```

Solution — Create a Virtual Network and Subnets

Reference: *Azure CLI reference* → *Network* → *az network vnet*: <https://learn.microsoft.com/cli/azure/network/vnet#az-network-vnet-create> (or `az network vnet create --help` / `az network vnet subnet`

create --help).

Address space versus subnet prefix

The VNet gets the **big** range (--address-prefixes); each subnet gets a **smaller** range inside it (--subnet-prefixes). `az network vnet create` can make the VNet and its first subnet in one call, so you create `web` with the VNet, then add `data` as a second subnet.

```
az group create -n lab-network-vnet01-rg -l centralus

az network vnet create -g lab-network-vnet01-rg -n lab-vnet \
  --address-prefixes 10.10.0.0/16 \
  --subnet-name web --subnet-prefixes 10.10.1.0/24

az network vnet subnet create -g lab-network-vnet01-rg --vnet-name lab-vnet \
  -n data --address-prefixes 10.10.2.0/24
```

Confirm the layout

```
az network vnet subnet list -g lab-network-vnet01-rg --vnet-name lab-vnet \
  --query "[].{name:name, prefix:addressPrefix}" -o table
# name  prefix
# web   10.10.1.0/24
# data  10.10.2.0/24
```

Both subnet ranges sit inside 10.10.0.0/16, the VNet's address space. Nothing here costs money yet; you will place a VM in the `web` subnet in Question 2.

Goal	Command
See VNet create flags	<code>az network vnet create --help</code>
Create VNet + first subnet	<code>az network vnet create -g <rg> -n <vnet> --address-prefixes 10.10.0.0/16 --subnet-name web --subnet-prefixes 10.10.1.0/24</code>
Add another subnet	<code>az network vnet subnet create -g <rg> --vnet-name <vnet> -n data --address-prefixes 10.10.2.0/24</code>
List subnets	<code>az network vnet subnet list -g <rg> --vnet-name <vnet> --query "[].{name:name, prefix:addressPrefix}" -o table</code>
Show the VNet's address space	<code>az network vnet show -g <rg> -n <vnet> --query addressSpace.addressPrefixes -o tsv</code>

Question 2 — Place a VM in a Subnet

Scenario

The virtual network from Question 1 is ready. Now the web tier needs a server. As the cloud administrator, deploy a Linux VM **into the web subnet** so it picks up a private IP from that subnet's range.

`setup.bash` provisions the VNet (`lab-vnet`) with the `web` and `data` subnets for you.

The idea

When you create a VM you tell Azure which VNet and subnet to attach its network card to. The VM then gets a **private IP** from that subnet's range. Put it in the `web` subnet (`10.10.1.0/24`) and its private IP will be a `10.10.1.x` address. If you forget the `--vnet-name/--subnet` flags, `az vm create` builds a brand-new network instead, which is the usual “why isn't it in my subnet?” mistake.

Requirements

Everything in `lab-network-vnet02-rg`:

#	Requirement	Value
1	A Linux VM attached to the <code>web</code> subnet	private IP inside <code>10.10.1.0/24</code>

After it boots, look at its private IP and confirm it lands in the `web` range. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash      # provisions the VNet + subnets; you create the VM
# ... create a Linux VM in the 'web' subnet, then check its private IP ...
./validate.bash
./cleanup.bash
```

Solution — Place a VM in a Subnet

Reference: *Azure CLI reference* → `az vm create`: <https://learn.microsoft.com/cli/azure/vm#az-vm-create> (or `az vm create --help`). The flags that matter here are `--vnet-name` and `--subnet`: they attach the VM to an *existing* network instead of making a new one.

Create the VM in the web subnet

```
az vm create -g lab-network-vnet02-rg -n web-vm \
  --image Ubuntu2404 --size Standard_B2als_v2 \
  --vnet-name lab-vnet --subnet web \
  --admin-username azureuser --generate-ssh-keys
```

`--generate-ssh-keys` creates a key pair (no password to set). The VM also gets a public IP by default so you can reach it. This takes about a minute.

Confirm the private IP

```
az vm list-ip-addresses -g lab-network-vnet02-rg -n web-vm \
  --query "[0].virtualMachine.network.privateIpAddresses[0]" -o tsv
# -> 10.10.1.4    (a 10.10.1.x address = it's in the web subnet)
```

Azure hands out the first few addresses of a subnet to its own services, so the first VM usually lands on .4. Any 10.10.1.x confirms the VM is in web.

Goal	Command
See VM create flags	az vm create --help
Create a VM in a subnet	az vm create -g <rg> -n <vm> --image Ubuntu2404 --size Standard_B2als_v2 --vnet-name <vnet> --subnet web --admin-username azureuser --generate-ssh-keys
Show private IP	az vm list-ip-addresses -g <rg> -n <vm> --query "[0].virtualMachine.network.privateIpAddresses[0]" -o tsv
Show public IP	az vm list-ip-addresses -g <rg> -n <vm> --query "[0].virtualMachine.network.publicIpAddresses[0].ipAddress" -o tsv

Question 3 — Secure a Subnet with an NSG

Scenario

A VM running a web server sits in the **web** subnet, but nobody can reach it: with no **network security group (NSG)**, Azure’s default rules deny inbound traffic from the internet. As the cloud administrator, add an NSG that permits SSH and HTTP, and apply it to the whole **web** subnet so every VM in that tier is governed by the same rules.

`setup.bash` provisions the VNet and the VM (nginx on port 80, no NSG).

The idea

An **NSG** is a stateful allow/deny list of traffic rules. You can attach it to a single VM’s network card or, better for a tier, to a **subnet** so every resource in that subnet inherits it. Each rule has a priority, a direction, a protocol, a source, and a destination port. You will allow inbound **22** (SSH) and **80** (HTTP) and let the default “deny everything else” stay in place.

Requirements

Everything in `lab-network-vnet03-rg`:

#	Requirement	Value
1	An NSG associated to the web subnet	allows inbound TCP 80 (and 22)

Before the change, `curl http://<vm-public-ip>` times out; after you associate the NSG, it returns 200. Watch that flip. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash      # provisions the VM (nginx, no NSG); prints its public IP
# ... curl it (blocked), create+associate the NSG, curl again (200) ...
./validate.bash
./cleanup.bash
```

Solution — Secure a Subnet with an NSG

Reference: *Azure CLI reference* → *az network nsg*: <https://learn.microsoft.com/cli/azure/network/nsg#az-network-nsg-create> (or `az network nsg create --help`, `az network nsg rule create --help`, and `az network vnet subnet update --help`).

Move 1 — see it blocked

```
PUBIP=$(az vm list-ip-addresses -g lab-network-vnet03-rg -n web-vm \
  --query "[0].virtualMachine.network.publicIpAddresses[0].ipAddress" -o tsv)

curl --max-time 5 -i "http://$PUBIP"    # -> times out: no NSG, default deny
```

Move 2 — create the NSG, add rules, attach to the subnet

Three steps: make the NSG, add allow rules, then associate it to the `web` subnet. An NSG already permits outbound and intra-VNet traffic by default, so you only add the inbound rules you want (lower `--priority` number = evaluated first).

```
az network nsg create -g lab-network-vnet03-rg -n web-nsg

az network nsg rule create -g lab-network-vnet03-rg --nsg-name web-nsg -n allow-ssh \
  --priority 1000 --direction Inbound --access Allow --protocol Tcp --destination-port-ranges 22

az network nsg rule create -g lab-network-vnet03-rg --nsg-name web-nsg -n allow-http \
  --priority 1010 --direction Inbound --access Allow --protocol Tcp --destination-port-ranges 80

az network vnet subnet update -g lab-network-vnet03-rg --vnet-name lab-vnet -n web \
  --network-security-group web-nsg
```

Move 3 — confirm it's reachable

```
curl --max-time 5 -i "http://$PUBIP" # -> HTTP/1.1 200 OK, "web-vm is serving on port 80"
```

The same request that timed out now returns the page: the subnet NSG opened port 80 for the whole tier. (Rule changes take effect within a few seconds.)

Goal	Command
Create an NSG	<code>az network nsg create -g <rg> -n <nsg></code>
Allow an inbound port	<code>az network nsg rule create -g <rg> --nsg-name <nsg> -n <rule> --priority <n> --direction Inbound --access Allow --protocol Tcp --destination-port-ranges 80</code>
Attach NSG to a subnet	<code>az network vnet subnet update -g <rg> --vnet-name <vnet> -n web --network-security-group <nsg></code>
List a subnet's NSG	<code>az network vnet subnet show -g <rg> --vnet-name <vnet> -n web --query networkSecurityGroup.id -o tsv</code>
List NSG rules	<code>az network nsg rule list -g <rg> --nsg-name <nsg> -o table</code>

Question 4 — Peer Two Virtual Networks

Scenario

Two teams run in two separate virtual networks, `vnet-a` (10.10.0.0/16) and `vnet-b` (10.20.0.0/16). By default the VNets are completely isolated: a VM in one cannot reach a VM in the other, even over private IPs. The teams now need to talk. As the cloud administrator, connect the two networks with **VNet peering**.

`setup.bash` provisions both VNets and a VM in each: `vm-a` (with SSH open, your way in) and `vm-b` (running nginx, private IP only).

The idea

Peering links two VNets so traffic routes between them over Azure's backbone, using private IPs, as if they were one network. Peering is **directional**: you create it on each side (a-to-b and b-to-a). Only when both halves exist does the connection report **Connected** and traffic actually flow.

Requirements

Everything in `lab-network-vnet04-rg`:

#	Requirement	Value
1	Peering <code>vnet-a</code> to <code>vnet-b</code>	state Connected
2	Peering <code>vnet-b</code> to <code>vnet-a</code>	state Connected

Prove it: from `vm-a`, `curl vm-b`'s private IP. It times out before peering and returns 200 after. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash      # provisions both VNets + vm-a and vm-b; prints their IPs
# ... from vm-a, curl vm-b (blocked) -> peer both directions -> curl again (200) ...
./validate.bash
./cleanup.bash
```

Solution — Peer Two Virtual Networks

Reference: Azure CLI reference → `az network vnet peering`: <https://learn.microsoft.com/cli/azure/network/vnet/peering#az-network-vnet-peering-create> (or `az network vnet peering create --help`).

Move 1 — see the isolation

`setup.bash` printed `vm-a`'s public IP and `vm-b`'s private IP. SSH into `vm-a` and try to reach `vm-b`:

```
ssh azureuser@<vm-a-public-ip>
curl --max-time 5 http://<vm-b-private-ip> # -> times out: the VNets are isolated
exit
```

Move 2 — peer both directions

Peering needs the *resource ID* of the remote VNet on each side. Grab both, then create the two halves:

```
AID=$(az network vnet show -g lab-network-vnet04-rg -n vnet-a --query id -o tsv)
BID=$(az network vnet show -g lab-network-vnet04-rg -n vnet-b --query id -o tsv)

az network vnet peering create -g lab-network-vnet04-rg -n a-to-b \
    --vnet-name vnet-a --remote-vnet "$BID" --allow-vnet-access

az network vnet peering create -g lab-network-vnet04-rg -n b-to-a \
    --vnet-name vnet-b --remote-vnet "$AID" --allow-vnet-access
```

After the second one, both report Connected:

```
az network vnet peering list -g lab-network-vnet04-rg --vnet-name vnet-a \
    --query "[].{name:name, state:peeringState}" -o table
# name      state
# a-to-b    Connected
```

Move 3 — confirm connectivity

```
ssh azureuser@<vm-a-public-ip>
curl --max-time 5 http://<vm-b-private-ip> # -> Hello from vm-b, reached over the peering
exit
```

The private IP that was unreachable a moment ago now answers. Note you never opened a public port on vm-b: peering carries the traffic privately, and vm-b's default rules already allow traffic from within its (now-peered) virtual network.

Goal	Command
Get a VNet's resource ID	az network vnet show -g <rg> -n <vnet> --query id -o tsv
Create one side of a peering	az network vnet peering create -g <rg> -n <name> --vnet-name <local> --remote-vnet <remote-id> --allow-vnet-access
List peerings + state	az network vnet peering list -g <rg> --vnet-name <vnet> --query "[].{name:name, state:peeringState}" -o table

Question 5 — Load-Balance Two VMs

Scenario

Two identical web servers, `vm1` and `vm2`, each serve the app. Right now there is no single address in front of them and no way to spread traffic or survive one VM failing. As the cloud administrator, put a **public load balancer** in front of both so one IP distributes requests across them.

`setup.bash` provisions the VNet and both VMs (nginx, each serving its own hostname, port 80 open).

The idea

A **load balancer** takes traffic on a **frontend** (a public IP + port) and spreads it across a **backend pool** of VMs, sending only to the ones a **health probe** says are healthy. You wire up four things: a frontend (public IP), a backend pool (the two VMs' network cards), a probe (is port 80 answering?), and a rule (frontend 80 to backend 80).

Requirements

Everything in `lab-network-vnet05-rg`:

#	Requirement	Value
1	A public load balancer with a rule	frontend port 80 to backend port 80
2	A health probe	on port 80
3	The backend pool contains both VMs	2 members

Prove it: `curl` the load balancer's public IP several times and watch the reply alternate between `vm1` and `vm2`. (Walkthrough in `solution.md`.)

Work the question

```
./setup.bash      # provisions vm1 + vm2 (nginx); you build the load balancer
# ... create the LB, pool, probe, rule; add both VMs; curl the LB IP repeatedly ...
./validate.bash
./cleanup.bash
```

Solution — Load-Balance Two VMs

Reference: *Azure CLI reference* → `az network lb`: <https://learn.microsoft.com/cli/azure/network/lb#az-network-lb-create> (or `az network lb create --help`, plus `az network lb probe create --help` and `az network lb rule create --help`).

Step 1 — create the load balancer, frontend, and backend pool

`az network lb create` builds the LB, a new public IP, the frontend, and an empty backend pool in one call:

```
az network lb create -g lab-network-vnet05-rg -n web-lb --sku Standard \
  --public-ip-address web-lb-pip \
  --frontend-ip-name frontend --backend-pool-name backend-pool
```

Step 2 — add a health probe and a rule

```
az network lb probe create -g lab-network-vnet05-rg --lb-name web-lb -n health-probe \
--protocol Http --port 80 --path /

az network lb rule create -g lab-network-vnet05-rg --lb-name web-lb -n http-rule \
--protocol Tcp --frontend-port 80 --backend-port 80 \
--frontend-ip-name frontend --backend-pool-name backend-pool --probe-name health-probe
```

Step 3 — put both VMs in the backend pool

The pool holds each VM's NIC ip-config. Discover the NIC and ip-config names, then add them:

```
for vm in vm1 vm2; do
  NIC_ID=$(az vm show -g lab-network-vnet05-rg -n "$vm" \
    --query "networkProfile.networkInterfaces[0].id" -o tsv)
  NIC=$(basename "$NIC_ID")
  IPCFG=$(az network nic ip-config list --nic-name "$NIC" -g lab-network-vnet05-rg \
    --query "[0].name" -o tsv)
  az network nic ip-config address-pool add -g lab-network-vnet05-rg \
    --nic-name "$NIC" --ip-config-name "$IPCFG" \
    --lb-name web-lb --address-pool backend-pool
done
```

Step 4 — confirm traffic spreads

```
LBIP=$(az network public-ip show -g lab-network-vnet05-rg -n web-lb-pip --query ipAddress -o tsv)

for i in $(seq 6); do curl -s "http://$LBIP"; echo; done
# Served by vm1
# Served by vm2
# Served by vm1
# ...
```

The replies alternate between the two VMs: one public address, traffic spread across both backends. (The probe only sends traffic to healthy VMs, so if you stopped `vm1`, every reply would come from `vm2`.)

Standard SKU note: a Standard load balancer only forwards traffic that an NSG explicitly allows, which is why `setup.bash` opened port 80 on the VMs.

Goal	Command
Create LB + public IP + pool	<pre>az network lb create -g <rg> -n <lb> --sku Standard --public-ip-address <pip> -- frontend-ip-name frontend --backend-pool- name backend-pool</pre>
Add a health probe	<pre>az network lb probe create -g <rg> --lb- name <lb> -n health-probe --protocol Http - -port 80 --path /</pre>
Add a rule	<pre>az network lb rule create -g <rg> --lb-name <lb> -n http-rule --protocol Tcp -- frontend-port 80 --backend-port 80 -- frontend-ip-name frontend --backend-pool- name backend-pool --probe-name health-probe</pre>

Goal	Command
Add a VM's NIC to the pool	<pre>az network nic ip-config address-pool add - g <rg> --nic-name <nic> --ip-config-name <ipcfg> --lb-name <lb> --address-pool backend-pool</pre>
Get the LB public IP	<pre>az network public-ip show -g <rg> -n <pip> --query ipAddress -o tsv</pre>

Part 3 — Practice Exercises

Practice Exercises — Networking

These are practice problems: **questions only**, no `setup.bash`, no `validate.bash`, no `solution.md`. The graded questions walked you through the core skills; these are the reps that turn understanding into muscle memory.

Work them the way a cloud admin works any ticket:

1. **Read the scenario into a checklist** of concrete constraints.
2. **Find the reference**: guess `az network <noun> <verb>`, confirm in the docs.
3. **Tame the flags with `--help`**, required first, then match the rest to your checklist.
4. **Build the command incrementally**, then **verify with your own eyes**.

Some questions deliberately leave a value for you to look up. Finding it in the docs is part of the exercise.

How to check your own work (no validator here)

You are the validator now. The commands that tell you the truth:

```
az network vnet subnet list -g <rg> --vnet-name <vnet> -o table
az network nsg rule list -g <rg> --nsg-name <nsg> -o table
az network vnet peering list -g <rg> --vnet-name <vnet> --query "[].{name:name,
↪ state:peeringState}" -o table
az network public-ip show -g <rg> -n <pip> --query "{ip:ipAddress,
↪ alloc:publicIPAllocationMethod}" -o table
az network route-table route list -g <rg> --route-table-name <rt> -o table
az vm list-ip-addresses -g <rg> -n <vm> -o table
```

Ground rules

- Use your **own resource groups**, a good convention is `practice-netNN-rg` so they are easy to find and delete. One group per exercise.
- Several of these create **real, billable VMs**. Clean up the moment you finish:

```
az group delete --name <your-rg> --yes --no-wait
```

- Region is your choice; if a size is not offered where you are, pick another region.

Exercise 1 — Design a three-tier network

An app has a public web tier, a private app tier, and a database tier, and each must sit in its own subnet.

Done when:

- A VNet exists with address space 10.30.0.0/16.
- Three subnets exist: **web** (10.30.1.0/24), **app** (10.30.2.0/24), and **data** (10.30.3.0/24).
- `az network vnet subnet list` shows all three with the right ranges.

Exercise 2 — Lock a subnet down with a service tag

The database subnet should accept traffic only from inside the virtual network, not from the internet, without you having to list IP ranges by hand.

Done when:

- An NSG is associated to a **data** subnet.
- It has an inbound rule allowing TCP 1433 whose **source is the VirtualNetwork service tag** (look up what a service tag is and which one means “this VNet and its peers”).
- Confirm with `az network nsg rule list` that the source is **VirtualNetwork**, not a raw IP range.

Exercise 3 — Steer traffic with a route table

You want all outbound traffic from a subnet to be sent somewhere specific (here, just prove you can install a custom route).

Done when:

- A **route table** exists and is associated to a subnet.
- It contains a route for destination 0.0.0.0/0 (look up the `--next-hop-type` options and pick **Internet** for this experiment).
- Confirm with `az network route-table route list`.

Exercise 4 — Static versus dynamic public IP

A service needs a public IP that never changes. Show the difference between the two allocation methods.

Done when:

- You create one public IP with **dynamic** allocation and one with **static**.
- You explain (to yourself) which one is guaranteed not to change and why that matters for DNS.
- Confirm the allocation method of each with `az network public-ip show --query publicIPAllocationMethod`.

Exercise 5 — Internal load balancer

Two backend VMs should be reachable through a single **private** address inside the VNet, with nothing exposed to the internet.

Done when:

- An **internal** (private) load balancer sits in a subnet with a private frontend IP (no public IP anywhere).
- Its backend pool holds two VMs, with a health probe and a rule on port 80.
- From a third VM in the same VNet, `curl http://<lb-private-ip>` returns a page, and repeated calls reach both backends.